

Assignment 3

Question 1 = Multimodal Learning

Date 06/04/2026

Problem 1

(a)

Given an embedding $e = Wx$, Euclidean distance

$$\|e_1 - e_2\|^2 = \|Wx_1 - Wx_2\|^2 = \|Wx_1 - Wx_2\|^2$$

Both encoders should apply the same weights W so embeddings are comparable.

$\Rightarrow e_1 - e_2 = W(x_1 - x_2)$, by removing squares.

Let $\Delta x = x_1 - x_2$, $e_1 - e_2 = W \cdot \Delta x$.

This is also the difference / distance, regardless of metric, between 2 embeddings.

$\Rightarrow e_1 - e_2 = W \cdot \Delta x = D_M$, where D_M is the distance under a certain modality M .

$$\Rightarrow (e_1 - e_2)^2 = (W \cdot \Delta x)^2 = D_M^2$$

Given Mahalanobis distance: $D_M^2 = (y - H\hat{z})^T S^{-1} (y - H\hat{z})$,

$$\Rightarrow (e_1 - e_2)^2 = (W \cdot \Delta x)^2 = (y - H\hat{z})^T S^{-1} (y - H\hat{z})$$

$$\Rightarrow (W \cdot \Delta x)^T (W \cdot \Delta x) = (y - H\hat{z})^T S^{-1} (y - H\hat{z})$$

$$\Rightarrow W^T \cdot \Delta x^T \cdot W \cdot \Delta x = (y - H\hat{z})^T S^{-1} (y - H\hat{z})$$

Since W encodes a raw measurement x to an embedding e , we can match terms:

$$\text{Data term: } \Delta x^T \cdot \Delta x = (y - H\hat{z})^T (y - H\hat{z})$$

Weight term: $W^T W = S^{-1}$ (it is a precision matrix, and also equivalent to the inverse of covariance).

$$\Rightarrow \text{Innovation covariance } S = (W^T W)^{-1}$$

$\therefore$ Given an effective matching, $\|e_1 - e_2\|^2 = D_M^2$.

(b)

In a linear case where $f_0(x) = e = Wx$, the innovation, represented by $y - \hat{y}$, only produces 1 uncertainty ellipse, with $\hat{y}$ at the center and the innovation forms the radius. But when f_0 is no longer linear, the innovation is also non-linear, (PI)

contd. Problem 1

Date 06/04/2026

(b) forming multiple ellipses, each of which represents an uncertainty (difference between the actual and predicted measurements) in a dimension.

Mathematically, if f_0 is linear, $f_0'(x)$ forms a constant. with $f_0 = Wx$, $f_0'(x) = W$. conversely, if f_0 is in higher order, for example $f_0 = W_1 x^2 + W_2 x$, f_0 could become an expression containing $x \Rightarrow f_0'(x) = 2W_1 x + W_2$, possibly in an addition of multiple terms, so the uncertainty has multiple constitutions.

In reality, one example is a captured object under motion. With a linear encoder, only pixels in a particular latent state are captured. But sometimes a sensor's weakness could not capture dynamics with noise accurately. A camera could record a moving car but not accurately with low light or with fog. A deep encoder could efficiently capture dynamics in non-linear dimensions. ✓

Because no single sensor is sufficient, most systems uses multiple ones and fuses signals. Another example is the data association determining contributions of signals from multiple sensors. With more dimensions, f_0 could encode richer weights to make the decision.

(P2)

Assignment 3

Contd. Problem 1

Date 06/04/2026

- (c) (i) $D_M^2(i, j)$ uses the Mahalanobis squared distance to show the difference between 2 detections, presented by the number of standard deviations between i and j . This is the first step to register measurements in a common metric.
- χ_{gate}^2 is a threshold determining what raw detections/measurements are being considered. This is the second step to gate detections.
- Then, embeddings e_i and e_j are obtained from raw detections i and j respectively. Embeddings represent features of a measurement. They are compared with the cosine similarity.
- $(1 - \cos(e_i, e_j))$ determines the dissimilarity between those two embeddings. By applying in the cost function, it serves as a data association step which determines which detections should be fused.
- Finally, both the gated detections and the associated features from measurements are weighted and fused.

Therefore, a concise flow has been formed:
Prediction $\rightarrow$ Registration $\rightarrow$ Gating $\rightarrow$ Representation (embedding) $\rightarrow$ Association $\rightarrow$ Fusion.

- (c) (ii) A different metric serves different purposes. Mahalanobis distance shows how many standard deviations a detection is apart from another. It concerns raw measurements. Cosine similarity shows

(P3)

Contd. Problem 1.

Date 06/04/2026

- (c) (ii) the angle between 2 vectors, which are embedded features. Those embeddings give meanings to a raw measurement in a specific context, and so, encoders that produce such an embedding are often trained to match compatible items. Both metrics balance the importances on raw detections and embedded features respectively for a modality.

With the Mahalanobis term alone where $\beta = 0$, it completely ignores semantic meanings of a detection, causing it not to accurately fit into context and associate the right predictions from detections.

Conversely, with only the cosine similarity where $\alpha = 0$, it completely ignores statistical frequentists' summary, for example, the ability to reject highly deviated samples or latent states. Those samples will remain and not be gated out properly. Even though we preserve the features from embeddings, at the first place, we will be taking too much or even all samples, leading to a high uncertainty (larger and more overlapping ellipses).

- (c) (iii) With a Mahalanobis distance showing the difference between 2 raw detections, x_{gate}^2 normalizes overly large D_M^2 values and highly deviated

(P4)

Assignment 3

contd. Problem 1

Date 06/04/2026

(c) (iii) samples. Without normalization, exploding values will occur, making the cost too large to be compared. It also suggests using all detections for further processes, from the modality. Larger sample size makes generalization difficult. Hence gating is necessary to control meaningful samples.

(c) (iv) α determines the reliance on Mahalanobis distance from raw detections, which is also normalized. β biases towards the importance of learned features. If the situation has less uncertainty, for example, cruising on a highway, we can put more reliance on α since it is simple and normalized. If the situation is more complex, and with more uncertainty, for example, continual stopping and going under traffic congestion, or under severe (like foggy) weather conditions where we need to associate signals from multiple modalities, we can put a higher weight on β for more reliance on the cosine similarity, which encapsulates learned features.

(d) (i) In JPDA, the marginal association probability

$$\beta_{ij} = \sum_{\theta: \theta_{ij}=1} P(\theta | y_t) \text{ at state } t$$

$$\Rightarrow \beta_{ij} = \sum_i \sum_j p(y_j | \text{track } i)$$

$$\Rightarrow \beta_{ij} \propto p(y_j | \text{track } i)$$

Having negative cost per track's observation means that each individual prediction probability per track

(P5)

Contd. Problem 1

Date 06/04/2026

(d) (i) is inversely proportional to the cost, which in turn encourages accuracy of a correct prediction based on an observed detection in order to lower the data association cost. From the formulation $C_{ij} = \alpha \cdot \frac{D_M^2(i,j)}{x_{gate}^2} + \beta (1 - \cos(e_i, e_j))$,

we are trying to reduce the deviation of an observation versus a prediction (at $D_M^2(i,j)$), as well as to minimize the difference between two learned feature embeddings (at $(1 - \cos(e_i, e_j))$).

(d) (ii) If all individual measurements from their corresponding tracks y_j | track i are independent events, with no correlations,

$$\beta_{ij} = \sum_i \sum_j p(y_j | \text{track } i) = \exp\left(-\frac{\alpha}{2} \frac{D_M^2}{x_{gate}^2}\right) \cdot \exp\left(-\frac{\beta}{2} (1 - \cos(e_i, e_j))\right)$$

This is according to independent probability events, where $P(A \cap B) = P(A) \cdot P(B)$ for any 2 random variables A and B .

If anything goes wrong, such that each y_j | track i is not independent, $P(A|B) \neq 0$ and $P(B|A) \neq 0$. There will also be a residual in $P(A \cap B)$ other than purely $P(A) \times P(B)$.

In JPDA, β_{ij} will be greater than the product of two exponentials.

In normal cases, β_j is mutually exclusive and exhaustive such that $\sum_{j=0}^P \beta_j = 1$.

(P6)

Assignment 3

Contd. Problem 1

Date 06/04/2026

(e) Hungarian algorithm is a hard assignment technique which matches the minimum weights to each y_j / track i , in polynomial time. In $C_{ij} = \alpha \frac{D_M^2(i, j)}{\chi_{gate}^2} + \beta(1 - \cos(e_i, e_j))$ in $O(n^3)$ time

to minimise α and β , given constant χ_{gate}^2 , we have to minimise each $D_M^2(i, j) = (y_j - H\hat{z}_i)^T S_i^{-1} (y_j - H\hat{z}_i)$ and $1 - \cos(e_i, e_j) = 1 - \frac{e_i \cdot e_j}{\|e_i\| \|e_j\|}$. This assignment

of α, β values happens per $(y_j$ / track i) detection. Hence, it is globally 1-to-1. Attention first calculates a score S_j per prediction j . Regardless the number of observations, it uses a dot product of query and key to be the score S_j . Then softmax simply calculates $\alpha_j = \frac{e^{S_j}}{\sum_k e^{S_k}}$, without

the need to look for each track of detection. Hence, it is not 1-to-1 assignment.

It is essential in multi-track tracking only when we want to know the cost per track as well. In general, the cost per prediction should suffice.

Problem 2

Date 07/04/2026

- (a) In the updating step, the Kalman gain $K = \frac{\Sigma_1}{\Sigma_1 + \Sigma_2}$, and $\Sigma_{t|t} = (1-K)\Sigma_1$ in any state t .

With this fused variance, we have:

$$\Sigma_{\text{fused}} = (1-K)\Sigma_1 = \left(1 - \frac{\Sigma_1}{\Sigma_1 + \Sigma_2}\right)\Sigma_1 = \frac{\Sigma_1 \Sigma_2}{\Sigma_1 + \Sigma_2}$$

Precision $\Lambda_t = \frac{1}{\Sigma_t}$ in any state t .

$$\begin{aligned} \Rightarrow \Lambda_{\text{fused}} &= \Sigma_{\text{fused}}^{-1} = \frac{\Sigma_1 + \Sigma_2}{\Sigma_1 \Sigma_2} \\ &= \frac{\Sigma_1}{\Sigma_1 \Sigma_2} + \frac{\Sigma_2}{\Sigma_1 \Sigma_2} \\ &= \frac{1}{\Sigma_1} + \frac{1}{\Sigma_2} \\ &= \Lambda_1 + \Lambda_2 \quad \square \end{aligned}$$

$$\mu_{\text{fused}} = \mu_1 + K \cdot \mu_2 = \mu_1 + \frac{\mu_2 \Sigma_1}{\Sigma_1 + \Sigma_2} //$$

- (b) The conditional independence suggests each observation $y^{(1)}$ and $y^{(2)}$ from an expert has no overlapping information, so that $p(y^{(1)}|z) \times p(y^{(2)}|z) = p(y^{(1)} \cap y^{(2)}|z)$. When assumption is violated, $y^{(1)} \cap y^{(2)} \neq \emptyset$. For example, in an autonomous vehicle, a radar detects a generic road condition while a camera (or multiple ones) captures specific situations clearer, or at a different angle. Even though they are both detecting from the same latent state z , their measurements $y^{(1)}$ and $y^{(2)}$ could be correlated such that summary

(P8)

Assignment 3

Contd. Problem 2

Date 07/04/2026

(b) information can be fused and described on a car's dashboard. In case of correlation, the independence assumption does not hold. If PoE is assumed, it could be overconfident of potential noises captured, perceiving that each of them works purely independent but in fact not. This results in $p(y^{(1)}, y^{(2)}|z) = p(y^{(1)}|z)p(y^{(2)}|z)$ double-counting shared information given $y^{(1)} \cap y^{(2)} \neq \emptyset$.

(c) confidence weighting $e_{\text{fused}} = \sum_m W_m e_m$ assigns a weight to each embedding from a modality's detection e_m . The weight shows how reliable the source is in order to fuse signals and embed them into e_{fused} . Weighting ensures $\sum_m W_m = 1$ so that we know contributions of each embedding e_m in the resulting one. The gate converts the weighted concatenation of each embedding e_m from each modality and applies a softmax function to ensure ranging $[0, 1]$, probabilistically. Similarly, the gate is then applied onto each e_m with an element-wise $\odot$ dot product operation. It fuses with the embedding feature-by-feature and also guarantees $\sum_m g_m = 1$ according to $e_{\text{fused}} = g \odot e_{\text{cam}} + (1-g) \odot e_{\text{rad}}$ in a 2-modal setting. Hence, it is a special case of confidence-weighted fusion by learning signals in a feature-based level.

(P9)

contd. Problem 2

Date 07/04/2026

(c) As derived in (a), the Kalman filter's gain updates the fused precision $\Lambda_{\text{fused}} = \Sigma_{\text{fused}}^{-1} = \Lambda_1 + \Lambda_2$ and $\Sigma_{\text{fused}} = \frac{\Sigma_1 \Sigma_2}{\Sigma_1 + \Sigma_2}$. According to PoE experts, latent state z is derived from $p(z|y^{(1)}, y^{(2)}) \propto p(z|y^{(1)}) \cdot p(z|y^{(2)})$. The Kalman gain K acts as a weighting factor from $y^{(1)}$ and $y^{(2)}$ respectively to $y^{(\text{fused})}$. If we match K to be W_g in the gated-fusion, it is clearly that $W_g = K = \frac{\Sigma_1}{\Sigma_1 + \Sigma_2} \Rightarrow W_g \propto \frac{1}{\Sigma_1 + \Sigma_2}$. When

$$\Lambda_{\text{fused}} = \Lambda_1 + \Lambda_2 = \Sigma_{\text{fused}}^{-1} = \frac{\Sigma_1 + \Sigma_2}{\Sigma_1 \Sigma_2},$$

$\Lambda_{\text{fused}} \propto \frac{1}{\Sigma_1 + \Sigma_2} \propto W_g$. Given $e_{\text{fused}} = g \odot e_{\text{cam}} + (1-g) \odot e_{\text{rad}}$, g is an individual weighting to e_m .

If $e_{\text{fused}} = \sum_m W_m e_m$, then $\sum_m g_m \odot e_m = \sum_m W_m e_m \Rightarrow$ so $g_m \propto W_m$ when both $\sum_m g_m = 1$ and in general $\sum_m W_m = 1$.

$$W_m = K_m = \frac{\Sigma_{m-1}^{-1}}{\Sigma_{m-1}^{-1} + \Sigma_m^{-1}} \Rightarrow W_m \propto \frac{1}{\Sigma_{m-1}^{-1} + \Sigma_m^{-1}}$$

$$\frac{1}{\Sigma_{m-1}^{-1} + \Sigma_m^{-1}} = \frac{1}{\Lambda_{m-1}^{-1} + \Lambda_m^{-1}} = \frac{1}{\Lambda_{\text{fused}}^{-1}} = \Lambda_{\text{fused}}$$

$$\Rightarrow \therefore W_m \propto \Lambda_{\text{fused}} \propto \Sigma_{\text{fused}}^{-1} \propto \Sigma_m^{-1}$$

$\Rightarrow \therefore W_m$, or g_m , is a learned, feature-level of the traditional confidence-weighted fusion. $\square$

(d) Mixture of Experts (MoE) fusion is defined summatively:

$$e_{\text{fuse}} = \sum_{k=1}^3 \alpha_k (W_g[e_1, \dots, e_k]) e_k$$

When an expert e_k collapses, the fused embedding

(P10)

Assignment 3

Contd. Problem 2

Date 07/04/2026

(d) will likely become too sparse. To regularize, one method is to apply Softplus with a newly introduced noise term:

$g = \sigma(W_g [e_{com}; e_{rad}]) + \epsilon \cdot \text{Softplus}(W_{noise} \cdot x)$, where $\epsilon \sim \mathcal{N}(0, 1)$. This method modifies the gate by introducing softplus, which enlarges the value with an exponential and takes a logarithm to control the range, such that: $f(x) = \ln(1 + e^x)$, and a random noise factor for stochasticity.

Another method is to regularize with a maximum entropy, which shows a degree of surprise. It applies to the gating too, such that: $H(g(x)) = - \sum_{i=1}^3 g_i(x) \log(g_i(x))$. If an expert behaves overly confident, its embedding is penalized by the entropy, via the gate function.

(e) If a modality (source of expert) outputs a sparse embedding with mostly zeros, it is a sign where it fails to capture the context due to a noisy environment, such as capturing geometry using LiDAR in a rainy condition. In this case, the gate should learn the patterns of a failed embedding. This could avoid its expert being overly confident and dominate the contribution onto the fused embedding. Naïve concatenation with MLP simply merges signals with equal importance and reliance. If no further embeddings

(P11)

Contd. Problem 2

Date 07/04/2026

(e) is augmented nor supplied before fusion, sparsity will also affect the fused embedding, such that the fusion will not be able to capture the true context precisely.

(f) When the third modality (LiDAR) is added, the gating becomes: $g = \sigma(W_g [e_{cam}; e_{rad}; e_{LiDAR}])$
the fused embedding becomes: $e_{fuse} = g_{cam} \odot e_{cam} + g_{rad} \odot e_{rad} + g_{LiDAR} \odot e_{LiDAR}$, where
 $g = g_{cam} + g_{rad} + g_{LiDAR} = 1$

In the two modality setting, $g_{fused} = \sum_{i=1}^2 g_i = 1$. There is one less component to decide the gating value that determines the reliability of embeddings probabilistically with softmax. No single modality is perfect to capture a real-world context due to noise. The more modalities are introduced, the reliability on each of them is reduced, so the fusion will be less likely affected by over confident and sparse embeddings, and thus, generalize the actual context better.

Assignment 3

Problem 3

Date 07/04/2026

- (a) JPDA with 2 tracks and 3 measurements:
Marginal Probability = $\sum_{i=1}^2 \sum_{j=1}^3 \beta_{ij}$
 $= \sum_{i=1}^2 \sum_{j=1}^3 \sum_{\theta: \theta(i)=j} P(\theta | Y_{1:k})$
Meanwhile, the marginal probability = $\sum_{i=1}^2 \beta_{ij} = 1$
 $\Rightarrow$ Number of β_{ij} components (joint events) = $2 \times 3 = 6$
- (b) α_{ij} performs a row-wise normalization with softmax, while β_{ij} is marginalized from all possible joint events without any preprocessing. Each score in the attention matrix is divided by a sum of its row. By softmax, the attention score (a weight) is constraint with a range of $[0, 1]$. But each row is independently normalized, so there is no constraint between rows. JPDA on the other hand, is more robust because it considers every possible probabilistic event in order to marginalize probabilities, and so, no missing detections could happen. Attention makes focuses by narrowing down measurements to ensure a query q_i from a track i matches closely with a key k_j . Thus, it could miss detections. Quality depends on its learning on the context to guarantee precise matching.
- (c) Sinkhorn normalizes each item in a $N \times M$ matrix row-wise and column-wise, so in turn it allows one more chance to cover values in the matrix. But the logit comes from $(q_i^T k_j / \sqrt{d})$, which is (P13)

Contd. Problem 3

Date 08/04/2026

(c) the score. The score is already a particular focus from the attention mechanism where q_i matches closely with k_j . With one more normalizing step, it could be closer to the optimal transport, which is theoretically a one-to-one strict Hungarian assignment. But it is still an approximation despite being closer. The complexity is $O(N \times M)$ by iterating through each value in the logit matrix by row then by column. To make the approximation even closer to the optimal transport, one could increase K , the number of iterations. The hard assignment happens after value assignment in the matrix. The resulting one is put in an exponent to give a doubly stochastic attention matrix.

(d) This is an output score from a MLP function/network: $l_{ij} = f_0(\varphi(z_i), \psi(y_j))$, by processing a latent state z_i , and its observation y_j . From an input layer to an output layer, a forward pass is shown below:

Let $u_i = \varphi(z_i)$ and $v_j = \psi(y_j)$

Attention weights (from input): $a_{ij} = W_1 u_i + W_2 v_j + b_1$

Hidden layer: $h_{ij} = \text{RELU}(a_{ij})$

output: $l_{ij} = W_3 \cdot h_{ij} + b_2$

where W_1, W_2, W_3, b_1 and b_2 are constants.

By chain rule: $\frac{\partial l_{ij}}{\partial \varphi(z_i)} = \frac{\partial l_{ij}}{\partial u_i} = \frac{\partial l_{ij}}{\partial h_{ij}} \cdot \frac{\partial h_{ij}}{\partial a_{ij}} \cdot \frac{\partial a_{ij}}{\partial u_i}$

(P14)

Assignment 3

Contd. Problem 3

Date 08/04/2026

$$(d) \Rightarrow \frac{\partial \ell_{ij}}{\partial \varphi(z_i)} = W_3 \cdot \text{RELU}(a_{ij}) \cdot W_1$$
$$\frac{\partial \ell_{ij}}{\partial \varphi(z_i)} = W_3 \cdot |W_1 \varphi(z_i) + W_2 \psi(y_j) + b_1| \cdot W_1$$

Note that RELU activation is a $\max(0, a_{ij})$ function which makes sure a_{ij} attention weights are non-negative.

With a simple dot-product attention,

$$\frac{\partial \ell_{ij}}{\partial \varphi(z_i)} = \frac{\partial}{\partial \varphi(z_i)} [f_0(\varphi(z_i), \psi(y_j))]$$
$$= \frac{\partial}{\partial \varphi(z_i)} [\varphi(z_i) \odot \psi(y_j)]$$
$$= \psi(y_j)$$

The gradient from the dot-product attention is simply $\psi(y_j)$, which means the gradient change of the attention score ℓ_{ij} with respect to the latent state $\varphi(z_i)$ scales directly and linearly with its observation $\psi(y_j)$.

Meanwhile, the MLP attention score is expressed with a constant multiplication from a RELU value. It transforms and normalizes negative attention weights. It also scales polynomially with W_3 and W_1 constants.

Therefore, MLP could handle more complex and perhaps non-linear attention weights, while dot product is simply expressed to scale linearly with an observation from the input, not even being focuses on particular observation items from a latent state with attention.

(P15)

Contd. Problem 3

Date 08/04/2026

(e) As shown in (a), the marginal probability of JPDA depends on number of tracks i and number of detections j , which also determines the number of additive terms β_{ij} . With a larger i and j , JPDA will have more terms and so, number of events $= i \times j$. Computation scales with $O(i \times j)$. Hungarian algorithm is a hard one-to-one assignment. It has to firstly look for all possible tracks i , and then to match an optimal detection j to each of them robustly. Thus, computation takes $O(i^2 j)$.

Multi-head attention is like the function in (d), which takes 2 pieces of information and finds a focus from a combination of the latent state z_i and its observation y_j . Everything scales linearly, from an input layer, to the RELU hidden layer, and an output layer. Thus, computation is $O(i)$, where each detection j is obtained from a state in a track i . Speaking of computation, multi-head attention is the cheapest data association method.

JPDA represents uncertainty at each $\beta_{ij} = P(\theta | Y_{1:k})$, which depends on the prior belief θ . Each β_{ij} is weighted by the innovation $\bar{v}_i = \sum_{j=1}^m \beta_{ij} v_{ij}$ where $v_{ij} = y_j - \hat{y}_{i,t}$, the difference between an actual observation and its prediction at a state i from Kalman filter. Thus, mean is updated in a way: $\hat{\mu}_{i,t} = \hat{\mu}_{i,t-1} + K_{i,t} \bar{v}_i$. Covariance is updated in a way: $\Sigma_{i,t} = \beta_{i,0} \Sigma_{i,t-1} + (1 - \beta_0) \Sigma_i^{KF} + K_{i,t} [\sum_j \beta_{ij} v_{ij} v_{ij}^T - \bar{v}_i \bar{v}_i^T] K_{i,t}^T$.

(P16)

Assignment 3

Contd. Problem 3

Date 09/04/2026

(e) Hungarian algorithm is deterministically robust so no uncertainty could be represented. A multi-head attention represents uncertainty through the loss $l_{ij} = f_{\theta}(\psi(z_i), \psi(y_j))$ for a certain latent state z_i and an observation y_j . The loss is minimized during backpropagation with chain rule representing the gradient flow. Speaking of the least uncertainty, Hungarian algorithm is the choice.

The gradient flow of JPDA is represented by a Jacobian $\frac{\partial}{\partial \theta} \beta_{ij}$ in each additive term. So it scales with

the marginal probability. While it is capped at 1, more additive terms will cause vanishing values of probabilities β_{ij} . Because a multi-head attention is derived in (d) that gradients are learned from

$$\frac{\partial}{\partial \psi(z_i)} = W_3 \cdot [W_1 \psi(z_i) + W_2 \psi(y_j) + b_i] \cdot W_1,$$

where $\psi(z_i)$ and $\psi(y_j)$ values are normalized with RELU and some weight terms W_1 and W_3 properly, it is the best option here.

In a real-time autonomous driving system, if the situation consists of high uncertainty frequently, for example, it always stay stuck in a traffic congestion, or weather conditions are not usually predictable, Hungarian algorithm is the most viable option.

Otherwise, especially in stable conditions like cruising on the highway, where uncertainty is guaranteed less, a multi-head attention provides focuses to a specific

(P17)

Contd. Problem 3

Date 09/04/2026

- (e) state z_i with respect to its observation y_j , and it is the cheapest option for data association with the simplest and most expressive gradient flow.

Problem 4

- (a) (i) To determine the best fusion taxonomy level, we need to know how closely related detections from both modalities should be. Camera and radar inputs work closely trying to form an understanding of a latent state. Because no single modality is sufficient, both signals must be fused to measure different contextual information. Late fusion occurring at the output must be the least viable option for cross-modalities. While early fusion takes raw signals directly and guarantees maximum cross modality interaction, mid fusion acts at a better position (at a feature's level) to obtain semantic and contextual features from input and thus, balances interaction and the learning of a particular modality. Hence, if signals are too noisy, like constantly changing weather conditions, I'd suggest mid fusion, or otherwise, early fusion to get maximum cross-modal interaction.

- (ii) At the input level of a transport (aircraft) system, it should be lightweight enough and expressively manages gradient flows for uncertainty management. Proven in problem 3 (e), a multi-head attention is the most viable option for data association, comparative to JPDA, and a hard Hungarian assignment of probabilities.

(P18)

Assignment 3

Contd. Problem 4

Date 09/04/2026

- (a) (iii) Multiple models handle temporal states. A diffusion model learns an observation from a state by progressively corrupt and reconstruct a latent state. It uses a score of a density $\nabla_x \log p(x)$ from a particular input sample x , to predict the next state with Langevin update: $x_{k+1} = x_k + \eta \nabla_x \log p(x) + \sqrt{2\eta} Z_k$, $Z_k \sim \mathcal{N}(0, I)$. It learns a full probability distribution with a deterministic drift.

Kalman Filter uses a prediction and an update step to get the next possible state. By setting up the model

$z_t = z_{t-1} + q_t$, where z_t is a latent state in state t and $q_t \sim \mathcal{N}(0, q_t)$; $y_t = z_t + r_t$, where y_t is the observation at state t and $r_t \sim \mathcal{N}(0, r_t)$.

From prediction, we measure the mean $\mu_{t|t-1} = \mu_{t-1|t-1}$ and variance $\sigma_{t|t-1}^2 = \sigma_{t-1|t-1}^2 + q_t$. In update, $\hat{y}_t$ is predicted to be $\mu_{t|t-1}$, innovation is defined $s_t = \sigma_{t|t-1}^2 + r_t$, and Kalman gain is $K_t = \frac{\sigma_{t|t-1}^2}{\sigma_{t|t-1}^2 + r_t}$, so,

$\mu_{t|t} = \mu_{t|t-1} + K_t (y_t - \hat{y}_t)$ and $\sigma_{t|t}^2 = (1 - K_t) \sigma_{t|t-1}^2$. It could estimate the exact optimal state. It is also very expressive for a Bayesian uncertainty quantification. Alternatively, particle filter is a Monte-Carlo sampling approach which gives us a weighted approximation of the most possible state.

An aircraft system would need precise temporal predictions, so the approximated particle filter is not the case. Diffusion model aims to learn via reconstruction. It is complex and does not favour real-

(19)

Contd. Problem 4

Date

09/04/2026

(a) time tracking and detection compared to a Kalman Filter. Hence, a Kalman Filter is the most viable option for a temporal model.

(iv). To compare uncertainty quantification approaches, we could explore fusion taxonomy levels in (i), association methods in (ii), and temporal models in (iii). Since the early fusion does not learn contextual features during fusion, and late fusion does not take cross-modal uncertainties into consideration, mid fusion could make the best balance and learn uncertainties from semantic embeddings. For association, it is derived in problem 3 (e) that a multi-head attention quantifies uncertainties in the most expressive and normalized gradient flow. While the Hungarian algorithm is robust already which needs not to quantify uncertainties, it is too costly for a real-time aircraft system. While both temporal models, a diffusion model and a Kalman Filter, can give a probabilistic/Bayesian estimate of the next time state, a diffusion model produces a deterministic drift from a Langevin update, and it is also too complex for a real-time system. A Kalman Filter expressively gives N_{ut} and σ_{ut} , which is perfect for uncertainty quantification.

(P20)

Assignment 3

Contd. Problem 4

Date 09/04/2026

- (b) Instead of using an Intersection over Union (IoU), mean Average Precision (mAP) is calculated from a center distance, which is a 2D bird's eye view Euclidean distance. The nuScenes Detection Score (NDS) is a weighted average of mAP and the mean true positive errors that measures the quality of true positive detections from 5 metrics.

$$NDS = \frac{1}{10} [5 \cdot mAP + \sum_{i=1}^5 (1 - \min(1, mTP_i))]$$

The per-class performance analyzes the strengths of identifying each class for a certain modality, and estimates the size, frequency and sensor characteristics. For example, LiDAR-based methods excel in large and dense objects like cars, while image-based modalities (like a camera) is well suited for detecting smaller or geometrically changing objects, like a moving bicycle. In machine learning terms, mAP can be interpreted as the overall detection's accuracy by an Euclidean distance. NDS relies on mTP, which takes account into true positives, and thus, provides estimates on the precision and recall while balancing with the accuracy. Since a BEV fusion is a top-down strategy, it should capture geometrical layouts of an object precisely. Per-class performance could help fusing signals from modalities of different strengths. However, BEV does not work well in late fusions because it sacrifices modality intersection for a maximum modularity, and thus, is good for detecting geometric details. For a C-UAS system with mid fusion, balancing

(P21)

Contd. Problem 4

Date 09/04/2026

(b) both intersection and modularity, per-class performance may be less useful. Instead, it needs mAP to evaluate and prioritize improving the overall accuracy and most importantly, NDS to evaluate geometric details for better modularity. A safety-critical system needs not to be overconfident from risks such as a high variance. With a general mAP metric alone seems too weak, so, a BEV in late fusion with solely mAP is not considered. To deploy a C-UAS system, 2 more metrics could be added: root mean squared error (RMSE) = $\sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$, where n is the sample size, y_i is the true observation and $\hat{y}_i$ is a predicted one, forming a squared innovation $(y_i - \hat{y}_i)$ as a result of minimizing potential risks in a safety critical system. Meanwhile, the continuous ranked probability score (CRPS), which takes a cumulative distribution function F and an observation x , is defined: $CRPS(F, x) = \int_{-\infty}^{\infty} (F(z) - I\{x \leq z\})^2 dz$, where $I\{x, z\}$ is an indicator function for $x \leq z$ or being 0 otherwise. Here, it acts as a proper scoring function that attains its minimum when the predictive distribution F and the data distribution are equal. It could be used for calibration, ensuring predicted probabilities to match empirical frequencies, and for sharpness, ensuring confidence in correct predictions while remaining unsure under uncertainty. When an accuracy-based mAP itself is not enough, a proper scoring rules helps calibration in risk-aware safety-critical systems.

(P22)

Assignment 3

Contd. Problem 4

Date 09/04/2026

- (c) An object detection involves 2 stages = classifying an object from an image and performing a multi-task regression to learn coordinates of a bounding box. A Bird's-Eye-View fusion is a top-down strategy that excels in learning an object's layout, like size and density, but struggles to learn geometric details. Split Conformal prediction uses a held-out calibration set that is separate from training a model with data samples. That held-out set is used to compute a non-conformity score s_i . If $\hat{q}$ forms a $(1-\alpha)$ quantile, from the labels distribution, it guarantees the label of any new sample to lie within a particular interval whose score must not exceed $\hat{q}$. It calibrates a predictor such that a marginal coverage of a class (from the image classification step) is guaranteed, regardless of a model's correctness. It is important for the predictor to rely on a correctly predicted class before using its belief to predict the bounding box region. Exchangeability assumption is hold such that data used for the conformal prediction must be i.i.d (independent identical distribution). It is important to have assumed. For example, in an autonomous vehicle, layouts of the road conditions (such as cars ahead and around), obtained from the BEV fusion, must be independent across time states, and follow an identical distribution, so as to generate a reliable, calibrated object detection outcome.

(P23)

Question 2 (Chosen Paper: Time Grad)

Problem 1

Date 10/04/2026

(a) $p_\theta(y) = p_\theta(x_{t_0:T}) = \prod_{t=t_0}^T p_\theta(x_t^\circ | h_{t-1})$, where the hidden layer is defined by encoding a time state with a recurrent neural network (RNN) and a condition: $h_t = \text{RNN}_\theta(\text{concat}(x_t^\circ, C_t), h_{t-1})$ and it starts with $h_0 = 0$. Model weights θ are learned by minimizing the negative log-likelihood (NLL). So, the condition c at a time state t is indeed learned within the RNN during training as a hidden state. $p_\theta(y|C) = p_\theta(x_{t_0:T}|C) = \prod_{t=t_0}^T \text{RNN}_\theta(\text{concat}(x_t^\circ, C_t), h_{t-1})$

(b) During training, model parameters θ are optimized by minimizing NLL: $\sum_{t=t_0}^T -\log p_\theta(x_t^\circ | h_{t-1})$, which depends on the previous hidden state. A hidden state h_t that is derived from a previous one h_{t-1} concatenates a state x_t° with a covariates matrix in a RNN. The NLL will appear different on each training step. The gradients will also change since the expectation $E_{x_t^\circ, \epsilon, n}[\|e - e_\theta(\sqrt{\bar{\alpha}_n} x_t^\circ + \sqrt{1 - \bar{\alpha}_n} \epsilon, h_{t-1}, n)\|^2]$ involves the hidden state too. Most importantly, there will be a loss of information. The conditions/covariates C_t is an embedding that captures additional time dependent embeddings (e.g., day of week, hour of day) and time independent ones, as well as lag features depending on the time frequency from our training data.

(c) Diffusion learns by corrupting a latent state with noise $\epsilon \sim \mathcal{N}(0, I)$ during training, followed by learning a distribution from the corruption within a reverse process.

(P24)

Question 2 (Chosen Paper: TimeGrad)

Contd. Problem 1

Date/0/04/2026

(c) The latent variable x^0 is corrupted and diffused. On the contrary, the hidden state h_{t-1} and covariates c_t are conditioned on x^0 during diffusion, and thus, they are not diffused. x^0 is the variable that we are trying to learn to recover from corruption, so, it must be diffused. Hidden states store the history of past time states. Those events are already completed and therefore, must remain deterministic. Covariates are fixed conditions and information that helps the RNN learn x^0 . Based on their roles in TimeGrad, they must be well distinguished.

(d) TimeGrad models a full predictive probability distribution, so it does not perform individual point predictions. TimeGrad is for time-series forecasting. To achieve the distribution $p_0(y) = \prod_{t=0}^T p_0(x_t^0 | h_{t-1})$, it captures a condition as a covariates matrix as part of the input for a RNN to learn from, so affirmatively it performs conditional density estimation. Of course, it also performs posterior sampling. An annealed Langevin dynamics process samples x_t^i to learn $x_t^{n-1} = \frac{1}{\sqrt{1-\bar{\alpha}_n}} (x_t^i - \frac{\beta_n}{\sqrt{1-\bar{\alpha}_n}} \epsilon_0(x_t^i, h_{t-1}, n)) + \sqrt{\Sigma_0} z$, giving $E_{x_t^i, \epsilon, n} [\| \epsilon - \epsilon_0(\sqrt{\bar{\alpha}_n} x_t^i + \sqrt{1-\bar{\alpha}_n} \epsilon, h_{t-1}, n) \|^2]$ with a Gaussian noise, $p(x^n | x^0) = \mathcal{N}(x^n; \sqrt{\bar{\alpha}_n} x^0, (1-\bar{\alpha}_n)I)$ and $p_0(x^n, n) = \frac{1}{\sqrt{1-\bar{\alpha}_n}} (x^n - \frac{\beta_n}{\sqrt{1-\bar{\alpha}_n}} \epsilon_0(x^n, n))$.

(P25)

Problem 2

Date 11/04/2026

- (a) TimeGrad's flow is as follows:
 context encoder $\rightarrow$ conditioning representation $\rightarrow$ denoiser
 $\rightarrow$ reverse diffusion sampling.

A context encoder translates a current state from a time series into embeddings. By employing covariates in a matrix form, for categorical features, it allows relationships within a category, or its context, to be captured when training time series models. With these fixed information, an autoregressive RNN forms a conditioning representation with a time state and a condition, and thus, forming a hidden layer/state. A denoiser ϵ_θ , where $\epsilon \sim \mathcal{N}(0, I)$, is trained in a forward process by taking gradient steps that eventually learn the best parameters θ . Finally, by sampling time states, a reverse process tries to learn the diffusion with a Gaussian noise in an annealed Langevin dynamics. By gradually reducing noise, it will ultimately reach a final generated time state. With the fixed covariates that we previously identified, we now have an external context of truth to rely on in order to guide a better conditioning forecast from a minimal loss.

- (b) The forward diffusion process admits sampling x^n at any arbitrary noise level n in closed form, with $\alpha_n = 1 - \beta_n$ and $\bar{\alpha}_n = \prod_{i=1}^n \alpha_i$. Then we can form:
 $p(x^n | x^0) = \mathcal{N}(x^n; \sqrt{\bar{\alpha}_n} x^0, (1 - \bar{\alpha}_n) I)$. In a RNN, training takes place by taking gradient steps, which

(P26)

Question 2 (Chosen Paper: TimeGrad)

Contd. Problem 2

Date 11/04/2026

- (b) tries to minimize the squared error between the denoiser ϵ_0 and an arbitrary noise $\epsilon \sim \mathcal{N}(0, I)$. The denoiser thus takes $\sqrt{\alpha_n} x_t^0 + \sqrt{1 - \alpha_n} \epsilon$, h_{t-1} (the previous hidden state) and $n \sim \text{Uniform}(1, \dots, N)$ in the forward process. As the reverse process tries to recover a corrupted (diffused) state, it thus takes x_t^1 , h_{t-1} , and n into the denoiser ϵ_0 , at every reverse step.

According to the reverse process, $x_t^{n-1} = \frac{1}{\sqrt{\alpha_n}} (x_t^1 - \frac{\beta_n}{\sqrt{1 - \alpha_n}} \epsilon_0(x_t^1, h_{t-1}, n)) + \sqrt{\Sigma_0} z$. It apparently has 2 additive terms. Initially, when $n = N$ in the for loop, x_t^1 has no previous state to rely on, so it must rely on the previous hidden state h_{t-1} computed by the RNN in the forward process. Epistemically, since it does not have enough prior information to proceed with the forecasting via reversing a corrupted state, it heavily relies on x_n and β_n , which are aleatoric noises and stochastic. Thus, at the beginning, conditioning is generally weaker. As the reverse process goes, it eventually comes to the final state where $n = 1 \Rightarrow z = 0$, so the $\sqrt{\Sigma_0} z$ is completely ignored. With sufficient information, the reverse process eventually becomes deterministic while generating the resulting x_t^0 , and stabilizes the output.

(P27)

Question 2 (Chosen Paper: TimeGrad)

Contd. Problem 2

Date 11/04/2026

(c) The RNN structure in the forward process aims to learn the conditional distribution of future steps of a multivariate time series given its past and covariates $C_{1:T}$, forming the probabilistic structure:

$$p_x(x_{t_0:T}^0 | x_{1:t_0-1}^0, C_{1:T}) = \prod_{t=t_0}^T p_x(x_t^0 | x_{1:t-1}^0, C_{1:T})$$

From the structure, it forms a Markov chain of i.i.d. events with a fixed condition on every step. While each future time step could have been introduced an increasing variance, it sets up the stage for the reverse process to learn Gaussian transitions starting with $p(x^N) = N(x^N; 0, I)$. So, the RNN is not just an arbitrary choice.

(d) First, the forward process of TimeGrad typically sets up the perfect stage to generate a Markov chain of future x_t^0 states. In an autoregressive RNN, it minimizes a Gaussian noise with the denoiser's output, which is for the reverse process to learn the true $x_{t_0}^0$ state eventually. So, it is definitely performing sequence modelling. Furthermore, each generated time-series state is connected in a Markov chain, and the reverse process learns the Gaussian transitions between them. So, theoretically, it also sets up a set/graph-like dependence between generated (hypothetical) states. And, unfortunately, it does not use any masked inference. The initiative is to solve $p_0(x^0) = \int p_0(x^{0:N}) dx^{1:N}$. This is intractable. Instead of computing a tractable likelihood with masks, TimeGrad uses conditioning. It passes fixed

(P28)

Question 2 (Chosen Paper: TimeGrad)

Contd. Problem 2

Date 11/04/2026

- (d) information as covariates in every state through a RNN into a denoiser ϵ_θ , and thus, minimizes the squared error with an arbitrary noise $\epsilon \sim \mathcal{N}(0, I)$ through gradient steps in the forward process. So, it does not use any masked inference here.

Problem 3

- (a) From the posterior in the KL divergences in the forward process, it is approximated in a tractable way: $p(x^{n-1} | x^n, x^0) = \mathcal{N}(x^{n-1}; \tilde{\mu}_n(x^n, x^0), \tilde{\beta} I)$, and it gives $\tilde{\mu}_n(x^n, x^0) = \frac{\sqrt{\alpha_{n-1}} \beta_n}{1 - \alpha_n} x^0 + \frac{\sqrt{\alpha_n} (1 - \alpha_{n-1})}{1 - \alpha_n} x^n$

and $\tilde{\beta}_n = \frac{1 - \alpha_{n-1}}{1 - \alpha_n} \beta_n$. Uncertainty is not directly

represented in the variance parameter, but through $\tilde{\beta}_n$, which is an arbitrary noise at level n when $\alpha_n = 1 - \beta_n$. With an aleatoric noise term, we can only approximate the uncertainty implicitly through sampling: $x_t^{n-1} = \frac{1}{\sqrt{\alpha_n}} \left(x_t^n - \frac{\beta_n}{\sqrt{1 - \alpha_n}} \epsilon_\theta(x_t^n, h_{t-1}, n) \right) + \sqrt{\Sigma_0} z$. We can interpret from the Σ_0 term.

- (b) (i) There are 3 ways to obtain a point estimate. First, one could apply $\mu \pm 2 \cdot \frac{\Sigma}{\sqrt{n}}$. It requires a mean of samples $\tilde{\mu}_n(x^n, x^0)$, total number of samples n , and a z -score. But z -score is implicit. Plus, it is a frequentist approach that does not quantify uncertainty. With a Bayesian estimation, we can either estimate the maximum

(P29)

Question 2 (Chosen Paper: TimeGrad)

Contd. Problem 3

Date 12/04/2026

(b)(i) likelihood (MLE) : $\hat{\theta}_{MLE} = \arg \max_{\theta} p(x^{1:N} | x^0, \theta)$
or the maximum a posteriori (MAP) : $\hat{\theta}_{MAP} = \arg \max_{\theta} p(\theta | x^{0:N}) = \arg \max_{\theta} \log p(x^{1:N} | x^0, \theta) + \log p(\theta)$,
where θ denotes a model's parameters as a belief.
However, TimeGrad aims to estimate a full predictive distribution. MAP only estimates the mode, which discards some information. The MLE seems robust, but evaluating $\arg \max_{\theta} p_{\theta}(x^0) = \int p_{\theta}(x^{0:N}) dx^{1:N}$ is intractable. Alternatively, the paper suggests that, for long time sequences, one could replace the RNN structure in TimeGrad with a transformer architecture. It could provide better conditioning. The model could also introduce an inductive bias. If relationships between entities are known, one could also incorporate a Graph Neural Network (GNN).

(b)(ii) When it comes to an "interval", we could identify whether a "confidence interval" or a "credible interval" is more suitable. A confidence interval is a frequentist method that estimates how likely data samples are fit into an arbitrary interval, and does not quantify uncertainty. A credible interval is a Bayesian method that estimates how likely a model fits those data samples given that they are confirmed to fit in an interval. As it takes a model's prior and posterior beliefs into consideration, it can quantify uncertainty. TimeGrad aims to estimate a full predictive probability distribution, so, constructing a credible interval is more appropriate. TimeGrad samples

(P 30)

Question 2 (Chosen Paper: Time Grad)

Contd. Problem 3

Date 12/04/2026

(b) (ii) from a complete time series history of the training data, and splits the contiguous sequence into a context window $[1, t_0)$ and a prediction interval $[t_0, T]$. Predictions are also visualized in a 50% and 90% interval respectively from the Traffic benchmarking dataset. Hence, it adopts sampling to approximate and visualize a 90% credible interval from a benchmark.

(b) (iii). We need a proper scoring rule, where the expected score is optimized when the predictive distribution equals the true conditional distribution, to obtain a calibration diagnostic. The paper uses the continuous ranked probability score (CRPS) for evaluating each time-series dimension. This could also be a post-hoc calibration. $CRPS(F, x) = \int_{-\infty}^{\infty} (F(z) - I(x \leq z))^2 dz$. It takes a continuous time state x and a cumulative density function (CDF) F . It gives a lower bound when the predictive CDF F equals the data distribution x . It generalizes an absolute-error from a point to a distribution. It is useful to predict full distributions with a Gaussian mean and variance, which perfectly aligns with TimeGrad. We could then employ an empirical F : $\hat{F}(z) = \frac{1}{S} \sum_{s=1}^S I\{x_s \leq z\}$, where I is an indicator function and S is the sample size, such that $x_s \sim F$. CRPS can be directly obtained from simulated samples of the conditional distribution at each time step. Finally, we can sum across CRPS scores across the D

(P31)

Contd. Problem 3

Date 12/04/2026

(b)(iii). time-series, both for the ground-truth data and sampled data, yielding $\hat{F}_{sum}(t)$ for each time step. So, $CRPS_{sum} = E_t [CRPS(\hat{F}_{sum}(t), \sum_i x_{i,t})]$. CRPS also implicitly assesses both the calibration (how well the predicted probabilities match observed frequencies) and the sharpness (how concentrated the distribution is).

(c). Diffusion theoretically corrupts a time-state with some noise and learns by reconstructing it, in order to perform confident time forecasting. Aleatoric uncertainty happens at those noises, which is irreducible with more data samples. While sampling x^n , the arbitrary noise level n is one of them. It yields $\alpha_n = 1 - \beta_n$ where $\bar{\alpha}_n = \prod_{i=1}^n \alpha_i$ to estimate $p(x^n | x^0) = \mathcal{N}(x^n; \sqrt{\bar{\alpha}_n} x^0, (1 - \bar{\alpha}_n) I)$. α_n and β_n are also aleatoric, producing aleatoric terms $\sqrt{\bar{\alpha}_n}$ in the Gaussian mean and $(1 - \bar{\alpha}_n)$ in the variance. They are also used in the gradient steps in the forward process to resemble and minimize the loss in Noise Conditional Score Networks.

An epistemic uncertainty could be reduced when more data samples come and regularizes an overly informative prior. From the same loss in Noise Conditional Score Networks, $x^{n-1} = \frac{1}{\sqrt{\bar{\alpha}_n}} (x^n - \frac{\beta_n}{\sqrt{1 - \bar{\alpha}_n}} \epsilon_\theta(x^n, n)) + \sqrt{\Sigma_\theta} Z$, the covariance block Σ_θ is reducible and thus, epistemic. Meanwhile, since the forward process in TimeGrad estimates a posterior distribution $p(x^{n-1} | x^n, x^0) = \mathcal{N}(x^{n-1}; \tilde{\mu}_n(x^n, x^0), \tilde{\beta}_n I)$, $\tilde{\mu}_n(x^n, x^0) = \frac{\sqrt{\bar{\alpha}_{n-1}} \beta_n}{1 - \bar{\alpha}_n} x^0 + \frac{\sqrt{\bar{\alpha}_n} (1 - \bar{\alpha}_{n-1})}{1 - \bar{\alpha}_n} x_n$

(P32)

Question 2 (Chosen Paper: TimeGrad)

Contd. Problem 3

Date 12/04/2026

- (c) increases when there are more additive terms from additional samples. With a higher mean, the model could become more confident in predictions, provided more data samples. Hence, despite with respect to noise factors α_n and β_n , $\tilde{x}_n$ is also epistemic.

TimeGrad obviously performs a Monte-Carlo based sampling to produce x_t^i in a reverse process via Langevin dynamics. It is an approximation to estimate and predict the true x_t^0 state by reconstructing diffused generated future states.

$$x_{t+1}^{i-1} \approx \frac{1}{\sqrt{\alpha_n}} \left(x_t^i - \frac{\beta_n}{\sqrt{1-\alpha_n}} \epsilon_\theta(x_t^i, h_{t-1}, n) + \sqrt{\Sigma_\theta} z \right)$$

This process is sampled and performed iteratively after a RNN produces hypothetical new time states until x_t^0 reached. Variability happens mainly at the diffuser ϵ_θ with the generated hidden states h_{t-1} from the forward process, not to mention aleatoric noises like α_n and β_n .

From the above justifications, the sampling variability is directly captured with a closed form in the reverse process, in an attempt to reconstruct diffused states by approximation.

- (d) calibration (with a proper scoring rule) evaluates the alignment of a predicted probability distribution to the true distribution according to the ground-truth.

(P33)

Question 2 (Chosen Paper: TimeGrad)

Contd. Problem 3

Date 12/04/2026

- (d) In TimeGrad, Monte-Carlo based sampling is used and evolves variability. In general, such types of sampling is just an approximated mean of a prediction. Although more data samples could make predictions tend to be more accurate, it does not guarantee fully aligned or calibrated. Uncertainty could still exist in those variability, like x_n and β_n in TimeGrad. In TimeGrad, more outputs mean more predicted states generated by the forward process. The model then learns to reconstruct from those predicted states to a true state. But the reconstructed outcome after the reverse process $x_i^{\dagger}$ is not automatically identical to the true state from the ground-truth. That is why CRPS is still needed to calibrate predictions. Therefore, having a model outputting 100 samples does not automatically give calibrated uncertainty.

Problem 4

- (a) TimeGrad minimizes an error within gradient steps in the forward process. Mean Absolute Error (MAE) takes an absolute value of the mean differences between a noise $\epsilon \sim \mathcal{N}(0, I)$ and a denoiser ϵ_{θ} . The advantage is its refusal of negative values which do not make sense as a "difference". However, it is not sufficient when values are too small. Similar, while the root mean squared error (RMSE) takes the square root of the squared mean from that difference, by rooting the value, it will still be

(P34)

Question 2 (Chosen Paper: TimeGrad)

Contd. Problem 4

Date 12/04/2026

(a) too small. Gradient steps will struggle to learn from vanishing values. That is why TimeGrad proposes using a squared error and introducing noise factors, such that the forward process takes gradient steps on $\nabla_0 \|e - e_0(\sqrt{\alpha_n} x_t^0) + \sqrt{1 - \alpha_n} \epsilon, h_{t-1}, n)\|^2$.

(b) For a continuous observation x and a predictive cumulative distribution function (CDF) F , CRPS, the continuous ranked probability score, is defined $CRPS(F, x) = \int_{-\infty}^{\infty} (F(z) - I(x \leq z))^2 dz$, where I is an indicator function returning $\begin{cases} 1 : x \leq z \\ 0 : \text{otherwise} \end{cases}$.

The ultimate goal is to evaluate the alignment between a predicted distribution and the true one. It penalizes observations whose embedding has a small and vanishing value ($x \leq z$ for any z), while it rewards normal observations. It is to avoid sparse observations that are likely from a measurement error. Like MAE, it also generalizes an observation point, from an absolute error, into a distribution. MAE takes an absolute value between the observed probability / prediction and the raw observation itself, to avoid negative values. The Brier score is calculated from: $\frac{1}{C} \sum_{c=1}^C (\hat{p}_c - I(x=c))^2$, where c is a class to evaluate and $\hat{p}_c$ is the predicted probability for observation x in class c . Like CRPS,

(P35)

Question 2 (Chosen Paper: TimeGrad)

Contd. Problem 4

Date 12/04/2026.

(b) Brier takes the squared difference to avoid negative value, but it also enlarges the difference and effectively avoids vanishing values, for easier learning in gradient steps.

(c) The difference between CRPS and Brier or MAE is the use of a cumulative distribution function rather than a single point's probability density. The latter is analytic because it clearly has a closed form. But it could not effectively model continuous sequences like time series. CRPS employs the empirical CDF of F , i.e., $\hat{F}(z) = \frac{1}{S} \sum_{s=1}^S I(X_s \leq z)$ with S samples $X_s \sim F$ as a natural approximation of the predictive CDF. This allows CRPS to directly be computed from simulated samples of the conditional distribution at each time point. And a summed CRPS value could be used to compare the alignment between the predictive and the true distribution.

(d) Each CRPS value in the summation is an empirical CDF from sampled observations. Each sample's cumulative probability is summed only if its observation state's value is large enough, enforced by $x > z$. The sum is then taken an expectation $E_t[\cdot]$ w.r.t. t , the time state. By taking mean, redundant values are counted once only, leaving no redundant scores per dimension. Each CRPS score is calculated from a sum of

(P36)

Question 2 (Chosen Paper: TimeGrad)

Contd. Problem 4

Date 12/04/2026

(d) an empirical CDF $\hat{F}_{\text{sum}}(t)$, with a sum of all predictions from a sample: $\sum_t x_{i,t}^o$. An observation x_t^o in a time state is multivariate, which has multiple observed features. A sum of those vectors captures dimensional features more effectively than a scalar $z \in \mathbb{R}$.

(e) (i) Since TimeGrad is mainly forecasting multivariate time states and observations that has more than one features, as a vector, it is common to explicitly model Gaussian distributions with a probabilistic model, like a Bayesian neural network. When it comes to probabilistic classifiers or any predictors, one could also use the negative log-likelihood as a standard proper scoring rule for evaluation. The loss can be calculated: $\mathcal{L}_{\text{NLL}} = -\log p_o(y|x)$, where x is an observation and y could be a predicted state, for instance, to evaluate the accuracy of state reconstruction. By minimizing NLL, we are finding the maximum likelihood (MLE) in Bayesian's terms. This flexibly handles univariate or multivariate observations.

(e) (ii) Time Series imputation is the process of replacing missing data samples, often represented as NaN values, with estimated values. As a post hoc evaluation method, other than CRPS, Brier scores

(P37)

Question 2 (Chosen Paper: TimeGrad)

Contd. Problem 4

Date

(e)(ii) also rely on an indicator function that filters out extremely sparse observations.

$$\text{Brier} = \frac{1}{n} \sum_{c=1}^C (\hat{p}_c - I(x=c))^2$$

It is an average among classes of the sum of squared errors between a predicted class's probability and an indication of a state x .

It only sums large enough observations, by interpreting sparsity to be 0.